Plugins

Quick Start

A plugin is simply a **small code module** that extends OpenClaw with additional functionality (commands, tools, and Gateway RPCs). Most of the time, you will use plugins when you require a feature that is not yet built into the core OpenClaw system (or when you wish to keep optional features separate from the main installation). Quick steps:

1. Check what is currently loaded:

```
openclaw plugins list
```

1. Install an official plugin (e.g., Voice Call):

```
openclaw plugins install @openclaw/voice-call
```

1. Restart the Gateway, then configure the plugin under `plugins.entries.<id>.config`.

Available Plugins (Official)

- As of January 15, 2026, Microsoft Teams is available exclusively as a plugin; if you use Teams, please install `@openclaw/msteams`. - Memory (Core) — Bundled memory search plugin (enabled by default via `plugins.slots.memory`)
- Memory (LanceDB) — Bundled long-term memory plugin (automatic recall/capture; set `plugins.slots.memory = "memory-lancedb"`)
- Voice Call — `@openclaw/voice-call`
- Zalo Personal — `@openclaw/zalouser`
- Matrix — `@openclaw/matrix`
- Nostr — `@openclaw/nostr`
- Zalo — `@openclaw/zalo`
- Microsoft Teams — `@openclaw/msteams`
- Google Antigravity OAuth (Provider Authentication) — Bundled as `google-antigravity-auth` (disabled by default)
- Gemini CLI OAuth (Provider Authentication) — Bundled as `google-gemini-cli-auth` (disabled by default)
- Qwen OAuth (Provider Authentication) — Bundled as `qwen-portal-auth` (disabled by default)
- Copilot Proxy (Provider Authentication) — Local VS Code Copilot Proxy bridge; distinct from the built-in `github-copilot` device login (bundled, disabled by default)

OpenClaw plugins are **TypeScript modules** loaded at runtime via jiti. **Configuration validation does not execute plugin code**; it relies on the plugin manifest and JSON Schema. See Plugin Manifest. Plugins can register:

- Gateway RPC methods
- Gateway HTTP handlers
- Agent tools
- CLI commands

- Background services
- Optional configuration validation
- **Skills** (by listing the `skills` directory in the plugin manifest)
- **Auto-reply commands** (executed without invoking the AI agent)

Plugins run **within the same process** as the Gateway; therefore, treat them as trusted code. Tool Authoring Guide: [Plugin Agent Tools].

Runtime Helpers

Plugins can access selected core helper utilities via `api.runtime`. For telephony TTS:

```
const result = await api.runtime.tts.textToSpeechTelephony({
text: "Hello from OpenClaw",
cfg: api.config,
});
```

Notes:

- Uses the core `messages.tts` configuration (OpenAI or ElevenLabs).
- Returns a PCM audio buffer + sample rate. Plugins must handle resampling/encoding for the specific provider.
- Edge TTS does not support telephony.

Discovery and Priority

OpenClaw scans in the following order:

1. Configuration Paths

- `plugins.load.paths` (files or directories)

1. Workspace Extensions

- `<workspace>/.openclaw/extensions/*.ts`
- `<workspace>/.openclaw/extensions/*/index.ts`

1. Global Extensions

- `~/.openclaw/extensions/*.ts`
- `~/.openclaw/extensions/*/index.ts`

1. Bundled Extensions (shipped with OpenClaw; **disabled by default**)

- `<openclaw>/extensions/*`

Bundled plugins must be explicitly enabled via `plugins.entries.<id>.enabled` or `openclaw plugins enable <id>`. Installed plugins are enabled by default but can be disabled in the same manner. Each plugin must include an `openclaw.plugin.json` file in its root directory. If a path points to a specific file, the plugin's root directory is the directory containing that file, which must also contain the manifest. If multiple plugins resolve to the same ID, the first match encountered in the order listed above takes precedence, and lower-priority duplicates are ignored.

Package Collections

A plugin directory can contain a `package.json` file featuring an `openclaw.extensions` field:

```
{
  "name": "my-pack",
  "openclaw": {
    "extensions": ["./src/safety.ts", "./src/tools.ts"]
  }
}
```

Each entry in this list becomes a distinct plugin. If a package lists multiple extensions, the plugin ID takes the format `name/<fileBase>`. If your plugins import npm dependencies, please install them within that directory to ensure the `node_modules` are available (`npm install` / `pnpm install`).

Channel Directory Metadata

Channel plugins can broadcast onboarding metadata via `openclaw.channel` and installation prompts via `openclaw.install`. This approach keeps the core directory free of data. Example:

```
{
  "name": "@openclaw/nextcloud-talk",
  "openclaw": {
    "extensions": ["./index.ts"],
    "channel": {
      "id": "nextcloud-talk",
      "label": "Nextcloud Talk",
      "selectionLabel": "Nextcloud Talk (self-hosted)",
      "docsPath": "/channels/nextcloud-talk",
      "docsLabel": "nextcloud-talk",
      "blurb": "Self-hosted chat via Nextcloud Talk webhook bots.",
      "order": 65,
      "aliases": ["nc-talk", "nc"]
    },
    "install": {
      "npmSpec": "@openclaw/nextcloud-talk",
      "localPath": "extensions/nextcloud-talk",
      "defaultChoice": "npm"
    }
  }
}
```

OpenClaw can also merge **external channel catalogs** (e.g., MPM registry exports). Place the JSON file in one of the following locations:

- `~/.openclaw/mpm/plugins.json`
- `~/.openclaw/mpm/catalog.json`
- `~/.openclaw/plugins/catalog.json`

Alternatively, set `OPENCLAW_PLUGIN_CATALOG_PATHS` (or `OPENCLAW_MPM_CATALOG_PATHS`) to point to one or more JSON files (separated by commas, semicolons, or `PATH` separators). Each file should contain `{ "entries": [ { "name": "@scope/pkg", "openclaw": { "channel": {...}, "install": {...} } } ] }`.

Plugin ID

Default Plugin ID:

- Package Collection: The `name` field in `package.json`
- Standalone File: The file's base name (`~/.../voice-call.ts` → `voice-call`)

If a plugin exports an `id`, OpenClaw will use it; however, it will issue a warning if this ID does not match the configured ID.

Configuration

```
{
  plugins: {
    enabled: true,
    allow: ["voice-call"],
    deny: ["untrusted-plugin"],
    load: { paths: ["~/Projects/oss/voice-call-extension"] },
    entries: {
      "voice-call": { enabled: true, config: { provider: "twilio" } },
    },
  },
}
```

Fields:

- `enabled`: Master switch (Default: `true`)
- `allow`: Allowlist (Optional)
- `deny`: Denylist (Optional; `deny` takes precedence)
- `load.paths`: Additional plugin files/directories
- `entries.<id>`: Per-plugin switch + configuration

Configuration changes **require a Gateway restart**. Validation Rules (Strict):

- Unknown plugin IDs within `entries`, `allow`, `deny`, or `slots` constitute an **error**.
- Unknown `channels.<id>` keys constitute an **error**, unless the plugin manifest declares the channel ID.
- Plugin configurations are validated using the JSON Schema embedded within `openclaw.plugin.json` (`configSchema`).
- If a plugin is disabled, its configuration is retained, and a **warning** is issued.

Plugin Slots (Exclusive Categories)

Certain plugin categories are **exclusive** (only one can be active at a time). Use `plugins.slots` to select which plugin occupies a given slot:

```
{
  plugins: {
    slots: {
      memory: "memory-core", // or "none" to disable memory plugins
    },
  },
}
```

If multiple plugins declare `kind: "memory"`, only the selected one is loaded. The others are disabled and accompanied by diagnostic information.

Control Interface (Schema + Labels)

The control interface utilizes `config.schema` (JSON Schema + `uiHints`) to render enhanced forms. At runtime, OpenClaw augments the `uiHints` based on the plugins it discovers:

- It adds per-plugin labels for `plugins.entries.<id>`, `.enabled`, and `.config`.
- It merges optional configuration field hints provided by plugins at the following locations: `plugins.entries.<id>.config.<field>`.

If you wish for your plugin's configuration fields to display user-friendly labels and placeholders (and to mark specific keys as sensitive), please provide both `uiHints` and a JSON Schema within your plugin manifest. Example:

```
{
  "id": "my-plugin",
  "configSchema": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "apiKey": { "type": "string" },
      "region": { "type": "string" }
    }
  },
  "uiHints": {
    "apiKey": { "label": "API Key", "sensitive": true },
    "region": { "label": "Region", "placeholder": "us-east-1" }
  }
}
```

CLI

```
openclaw plugins list
openclaw plugins info <id>
openclaw plugins install <path>                 # copy a local file/dir into
~/.openclaw/extensions/<id>
openclaw plugins install ./extensions/voice-call # relative path ok
openclaw plugins install ./plugin.tgz           # install from a local tarball
openclaw plugins install ./plugin.zip           # install from a local zip
openclaw plugins install -l ./extensions/voice-call # link (no copy) for dev
openclaw plugins install @openclaw/voice-call # install from npm
openclaw plugins update <id>
openclaw plugins update --all
openclaw plugins enable <id>
openclaw plugins disable <id>
openclaw plugins doctor
```

`plugins update` applies only to npm installations tracked under `plugins.installs`. Plugins may also register their own top-level commands (e.g., `openclaw voicecall`).

Plugin API (Overview)

Plugins export one of the following:

- A function: `(api) => { ... }`
- An object: `{ id, name, configSchema, register(api) { ... } }`

Plugin Hooks

Plugins can include hooks and register them at runtime. This allows plugins to bundle event-driven automations without the need to install separate hook packages.

Example

```
import { registerPluginHooksFromDir } from "openclaw/plugin-sdk";

export default function register(api) {
  registerPluginHooksFromDir(api, "./hooks");
}
```

Notes:

- Hook directories follow the standard hook structure (`HOOK.md` + `handler.ts`).
- Hook eligibility rules still apply (OS/binary/environment/configuration requirements).
- Hooks managed by plugins appear as `plugin:<id>` in the `openclaw hooks list` output.
- You cannot enable or disable plugin-managed hooks directly via `openclaw hooks`; instead, you enable or disable the plugin itself.

Provider Plugins (Model Authentication)

Plugins can register **model provider authentication** workflows, allowing users to complete OAuth or API key setup directly within OpenClaw (without the need for external scripts). Providers are registered via `api.registerProvider(...)`. Each provider exposes one or more authentication methods (OAuth, API keys, device codes, etc.). These methods power the following command:

- `openclaw models auth login --provider <id> [--method <id>]`

Example:

```
api.registerProvider({
  id: "acme",
  label: "AcmeAI",
  auth: [
    {
      id: "oauth",
      label: "OAuth",
      kind: "oauth",
      run: async (ctx) => {
        // Run OAuth flow and return auth profiles.
        return {
          profiles: [
            {
              profileId: "acme:default",
              credential: {
                type: "oauth",
                provider: "acme",
                access: "...",
                refresh: "...",
```

```
                expires: Date.now() + 3600 * 1000,
              },
            },
          ],
          defaultModel: "acme/opus-1",
        };
      },
    },
  ],
});
```

Notes:

- `run` receives a `ProviderAuthContext` containing the `prompter`, `runtime`, `openUrl`, and `oauth.createVpsAwareHandlers` utility helpers.
- Return a `configPatch` when you need to add default model or provider configurations.
- Return a `defaultModel` so that `--set-default` can update the Agent's default values.

-

Registering Message Channels

Plugins can register **channel plugins**, which behave similarly to built-in channels (WhatsApp, Telegram, etc.). Channel configurations are located under `channels.<id>` and are validated by your channel plugin code.

```
const myChannel = {
  id: "acmechat",
  meta: {
    id: "acmechat",
    label: "AcmeChat",
    selectionLabel: "AcmeChat (API)",
    docsPath: "/channels/acmechat",
    blurb: "demo channel plugin.",
    aliases: ["acme"],
  },
  capabilities: { chatTypes: ["direct"] },
  config: {
    listAccountIds: (cfg) => Object.keys(cfg.channels?.acmechat?.accounts ??
{}),
    resolveAccount: (cfg, accountId) =>
      cfg.channels?.acmechat?.accounts?.[accountId ?? "default"] ?? {
        accountId,
      },
  },
  outbound: {
    deliveryMode: "direct",
    sendText: async () => ({ ok: true }),
  },
};

export default function (api) {
  api.registerChannel({ plugin: myChannel });
}
```

Important Notes:

- Place your configuration under `channels.<id>` (rather than `plugins.entries`).
- `meta.label` is used for the labels displayed in CLI/UI lists.
- `meta.aliases` adds alternative IDs used for normalization and CLI input.
- `meta.preferOver` lists the IDs of channels to be skipped for auto-activation when both are configured.
- `meta.detailLabel` and `meta.systemImage` allow the UI to display richer channel labels and icons.

Creating a New Messaging Channel (Step-by-Step Guide)

Use this method when you want to create a **new chat interface** ("messaging channel") rather than a model provider. Documentation for model providers can be found under `/providers/*`.

1. Select an ID + Configuration Structure

- All channel configurations reside under `channels.<id>`.
- For multi-account setups, prioritize using `channels.<id>.accounts.<accountId>`.

1. Define Channel Metadata

- `meta.label`, `meta.selectionLabel`, `meta.docsPath`, and `meta.blurb` control how the channel appears in CLI/UI lists.
- `meta.docsPath` should point to a documentation page, such as `/channels/<id>`.
- `meta.preferOver` allows your plugin to supersede another channel (ensuring it is automatically selected first).
- `meta.detailLabel` and `meta.systemImage` are used by the UI for displaying detailed text and icons.

1. Implement Required Adapters

- `config.listAccountIds` + `config.resolveAccount`
- `capabilities` (chat types, media support, threading, etc.)
- `outbound.deliveryMode` + `outbound.sendText` (for basic message sending)

1. Add Optional Adapters as Needed

- `setup` (setup wizards), `security` (private messaging policies), `status` (health/diagnostics)
- `gateway` (start/stop/login), `mentions`, `threading`, `streaming`
- `actions` (message actions), `commands` (native command behaviors)

1. Register the Channel within the Plugin

- `api.registerChannel({ plugin })`

Minimal Configuration Example:

```
{
  channels: {
    acmechat: {
      accounts: {
        default: { token: "ACME_TOKEN", enabled: true },
      },
    },
  },
}
```

Minimum Channels Plugin (Outbound Only):

```
const plugin = {
  id: "acmechat",
  meta: {
    id: "acmechat",
    label: "AcmeChat",
    selectionLabel: "AcmeChat (API)",
    docsPath: "/channels/acmechat",
    blurb: "AcmeChat messaging channel.",
    aliases: ["acme"],
  },
  capabilities: { chatTypes: ["direct"] },
  config: {
    listAccountIds: (cfg) => Object.keys(cfg.channels?.acmechat?.accounts ??
{}),
    resolveAccount: (cfg, accountId) =>
      cfg.channels?.acmechat?.accounts?.[accountId ?? "default"] ?? {
        accountId,
      },
  },
  outbound: {
    deliveryMode: "direct",
    sendText: async ({ text }) => {
      // deliver `text` to your channel here
      return { ok: true };
    },
  },
};

export default function (api) {
  api.registerChannel({ plugin });
}
```

Load the plugin (from the extensions directory or `plugins.load.paths`), restart the Gateway, and then configure `channels.<id>` within the configuration settings.

Agent Tools

Refer to the dedicated guide: [Plugin Agent Tools](#).

Register Gateway RPC Methods

```
export default function (api) {
  api.registerGatewayMethod("myplugin.status", ({ respond }) => {
    respond(true, { ok: true });
  });
}
```

Register CLI Commands

```
export default function (api) {
  api.registerCli(
    ({ program }) => {
      program.command("mycmd").action(() => {
        console.log("Hello");
      });
    },
    { commands: ["mycmd"] },
  );
}
```

Registering Automated Response Commands

Plugins can register custom slash commands that execute **without invoking an AI agent**. This is useful for toggle commands, status checks, or quick actions that do not require LLM processing.

```
export default function (api) {
  api.registerCommand({
    name: "mystatus",
    description: "Show plugin status",
    handler: (ctx) => ({
      text: `Plugin is running! Channel: ${ctx.channel}`,
    }),
  });
}
```

Command Handler Context:

- `senderId`: The sender's ID (if available)
- `channel`: The channel where the command was sent
- `isAuthorizedSender`: Whether the sender is an authorized user
- `args`: Arguments passed after the command (if `acceptsArgs: true`)
- `commandBody`: The full text of the command
- `config`: The current OpenClaw configuration

Command Options:

- `name`: The command name (without the leading `/`)
- `description`: The help text displayed in the command list
- `acceptsArgs`: Whether the command accepts arguments (Default: `false`). If `false` and arguments are provided, the command will not match, and the message will be passed to other handlers.
- `requireAuth`: Whether the sender must be authorized (Default: `true`)
- `handler`: A function that returns `{ text: string }` (can be asynchronous)

Example with Authorization and Arguments:

```
api.registerCommand({
  name: "setmode",
  description: "Set plugin mode",
  acceptsArgs: true,
  requireAuth: true,
  handler: async (ctx) => {
    const mode = ctx.args?.trim() || "default";
    await saveMode(mode);
    return { text: `Mode set to: ${mode}` };
  },
});
```

Important Notes:

- Plugin commands are processed **before** built-in commands and AI agents.
- Commands are registered globally and apply to all channels.
- Command names are case-insensitive (`/MyStatus` matches `/mystatus`).
- Command names must begin with a letter and may only contain letters, numbers, hyphens, and underscores.
- Reserved command names (e.g., `help`, `status`, `reset`, etc.) cannot be overridden by plugins.
- Duplicate command registrations across different plugins will fail and display a diagnostic error.
- Registering Background Services

```
export default function (api) {
  api.registerService({
    id: "my-service",
    start: () => api.logger.info("ready"),
    stop: () => api.logger.info("bye"),
  });
}
```

Naming Conventions

- Gateway Methods: `pluginId.action` (e.g., `voicecall.status`)
- Tools: `snake_case` (e.g., `voice_call`)
- CLI Commands: `kebab-case` or `camelCase`, but avoid conflicts with core commands.

Skills

Plugins may ship with associated Skills located within their repository (`skills/<name>/SKILL.md`). Enable them using `plugins.entries.<id>.enabled` (or other configuration toggles), and ensure that the Skill file is present in your workspace or designated Skills directory.

Distribution (npm)

Recommended Packaging Strategy:

- Core Package: `openclaw` (this repository)
- Plugins: Independent npm packages under the `@openclaw/*` scope (e.g., `@openclaw/voice-call`)

Publishing Contract:

- A plugin's `package.json` *must* include an `openclaw.extensions` field specifying one or more entry files.
- Entry files may be either `.js` or `.ts` (jiti is used to load TypeScript files at runtime).
- The `openclaw plugins install <npm-spec>` command executes `npm pack`, extracts the package contents to `~/.openclaw/extensions/<id>/`, and automatically enables the plugin within the configuration.
- Configuration Key Stability: Scoped packages are normalized to an **unscoped** ID for use within the `plugins.entries.*` configuration keys. ## Example Plugin: Voice Call

This repository contains a voice call plugin (supporting Twilio or a log-based fallback):

- Source Code: `extensions/voice-call`
- Skills: `skills/voice-call`
- CLI: `openclaw voicecall start|status`
- Tool: `voice_call`
- RPC: `voicecall.start`, `voicecall.status`
- Configuration (Twilio): `provider: "twilio"` + `twilio.accountSid/authToken/from` (optional: `statusCallbackUrl`, `twimlUrl`)
- Configuration (Dev): `provider: "log"` (offline mode)

Refer to Voice Call and `extensions/voice-call/README.md` for setup and usage instructions.

Security Considerations

Plugins run within the same process as the Gateway. Treat them as trusted code:

- Only install plugins that you trust.
- Prioritize using the `plugins.allow` allowlist.
- Restart the Gateway after making changes.

Testing Plugins

Plugins can (and should) be accompanied by tests:

- Plugins located within this repository can keep their Vitest tests under `src/**` (e.g., `src/plugins/voice-call.plugin.test.ts`).
- Independently published plugins should run their own CI (lint/build/test) and verify that `openclaw.extensions` points to the built entry point (`dist/index.js`).

For more information regarding plugins, please refer to: Tools and Plugins - OpenClaw